002- Neural Network

Difficulty: Medium Expect Time: 50 min Author: Cliff, Su

Neural networks are computational models designed to mimic the human brain. These models consist of layers that process information to recognize patterns and make predictions. Below is an explanation of the key components and their brief explanation:

- **Neural Network Layers**

  Neural network layers are the fundamental building blocks of artificial neural networks, designed to process and transform data progressively. Here’s a brief breakdown:

  1. **Input Layer**: First layer that receives data. It doesn’t perform computations.
  2. **Hidden Layers**: Layers perform mathematical transformations on the input data
  3. **Output Layer**: Final layer of the network, which produces predictions.

- **Optimizers**

  Optimizers are algorithms used in neural networks to adjust the model’s weights and minimize the error function efficiently during training. They determine how the network learns by updating weights based on gradients.

- **Error Functions**

  Error functions measure how far a neural network's predictions are from the actual values. They guide the optimization process, helping the model improve over time.

- **Gradient and Training (Forward and Backpropagation)**

  Neural networks improve their predictions through **training**, which involves adjusting weights using **gradient-based optimization**. The two main steps in training are **forward propagation** and **backpropagation**.

  - **Forward Propagation:**
    Computes predictions by passing inputs through the network layers.

  - **Backpropagation**:
    Calculates gradients for weights and biases using the chain rule, determining how much each parameter affects the error.

  - **Epochs**:
    An epoch is one complete pass through the entire dataset during training. Multiple epochs are required for the model to converge to an optimal solution.

- **Requirements**

1 Complete the following abstract classes by tracing the inherited classes:

- class Layer (**`Layer.h`**, inherited by: **`class Dense, class ReLU, class Sigmoid`**)
  attributes:
  - **`name`**

  member functions:
  - **`saveToString`**
  - **`loadFromString`**
  - **`getWeights`**
  - **`getBiases`**
  - **`getGradWeights`**
  - **`getGradBiases`**
  - **`forward`**
  - **`backward`**
  - **`print`**
- class Optimizer (**`Optimizer.h`**, inherited by: **`class Adam, class SGD`**)
  attributes:
  - **`learningRate`**

  member functions:
  - **`saveToString`**
  - **`loadFromString`**
  - **`update`**
  - **`print`**

1.1 You only need to modify the **`Layer.h`** and **`Optimizer.h`**. Once you complete them correctly, the main.cpp should runs successfully, training the model and saving the weights automatically. The model weights will be saved to **`{DATASET_NAME}_model.h`**, DO NOT MODIFY THE FILE.

2 Training the Model, all relevant code and model structures are included in the project, all you need is to:

- Replace DATASET_NAME with the 5 datasets commented in **`main.cpp`**
- Adjust the learning rate & epochs as needed(recommended values are commented on in **`main.cpp`**) and specify which optimizer to use (SGD/Adam).
  - If your trained model loss does not meet the requirement, we recommend you to
    1. Train with SGD optimizer with 1000 epochs first
    2. If the model still cannot be trained properly, try Adam optimizer with 100 epochs.
- You don’t need to worry about the loss function; it is fixed as MSE.

3 Evaluation Criteria

- You must ensure the training loss meets the required thresholds for the respective datasets:
  - cosine: Loss < 0.01
  - iris: Loss < 0.015
  - housing: Loss < 0.04
  - human_age: Loss < 0.01
  - cancer_type: Loss < 0.04

Input

1. Complete the required class implementation. The following files already contain some implementations
   a) **`Layer.h`**
   b) **`Optimizer.h`**
   You can submit a header-only implementation, or split the implementation into **`*.cpp`** files.
2. The input and output will be handled by the provided code.
3. The Online Judge will replace the following files:
   a） **`main.cpp`**
   b） **`core.h`**
4. The following files are part of the test cases of Online Judge. Please copy the contents of the following file into **`main.cpp`** for testing.
   a） **`case1.h`**

Output

1. Please DON’T print any data to STDOUT.
2. Training process and validation information of the model. Final loss of the model is showed at the bottom of output.
3. The sample output and the corresponding input files are shown as follows:
   a) **`output1.txt`** corresponds to **`case1.h`**

Other Notes

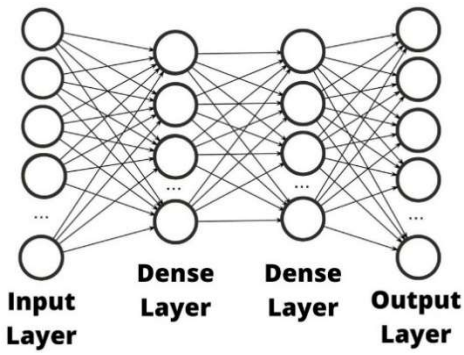
In the context of this exam, you are instructed to train and validate the model solely on the training set. While this approach simplifies the workflow for the purpose of learning, it is important to recognize that this method can lead to **overfitting**, where the model performs well on the training set but fails to generalize to unseen data.
In practice, datasets are usually divided into three subsets:

1. **Training Set**: Used to train the model, enabling it to learn patterns and relationships within the data.
2. **Validation Set**: Used during training to monitor and fine-tune the model's performance, helping to prevent overfitting.
3. **Test Set**: Used to evaluate the model's performance on completely unseen data to ensure it generalizes well to new inputs.

By employing this three-part split, the model's ability to generalize to real-world data can be assessed effectively. For this exam, the focus remains on training concepts and implementation, but you should understand that validation and testing on separate subsets are crucial in real-world applications.

Below are some examples of components of Neural Networks with real calculations:

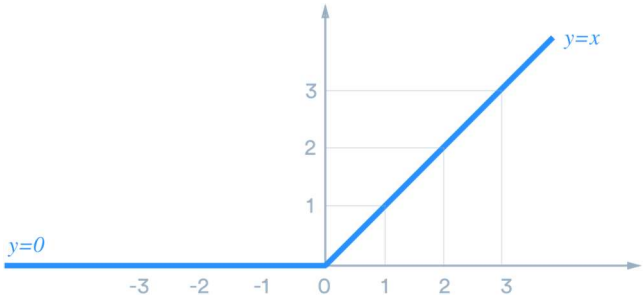
- **Neural Network Layers**
  The following are some common Neural Network Hidden Layers.
  - **Dense (Fully Connected Layer)**:
    Each neuron in this layer is connected to every neuron in the previous layer. It applies a weighted sum of inputs and adds a bias before passing the result through an activation function.



    - **Example Calculation:**
      Suppose a Dense layer has two inputs: x_1=1.5 and x_2=2.0, with weights w_1=0.8, w_2=-0.5, and bias b=0.2. The output z of the neuron is calculated as:

$$z = (x1 \cdot w1) + (x2 \cdot w2) + b = (1.5 \cdot 0.8) + (2.0 \cdot -0.5) + 0.2 = 1.2 - 1.0 + 0.2 = 0.4$$

- **ReLU (Activation Layer)**:
  Activation functions like ReLU introduce non-linearity, enabling the network to learn complex patterns.



  - **Example Calculation**:
    Using the previous z = 0.4, the ReLU output ReLU(z) is:

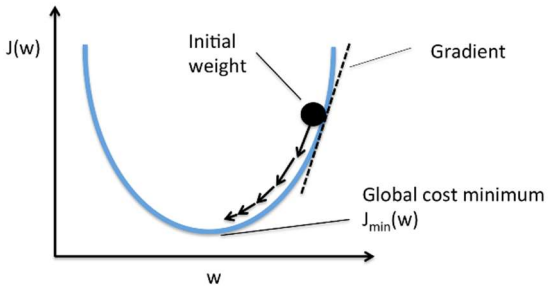
$$ReLU(z) = max(0, z) = max(0, 0.4) = 0.4$$

➢ **Optimizers**

The following are some common optimizers.

- **SGD (Stochastic Gradient Descent)**:
  Updates weights iteratively based on the gradient of the error for small, random subsets of data. It is computationally efficient but may converge slowly.

$$w = w - \eta \cdot \frac{\partial Error}{\partial w}$$



  - **Example Calculation**:

    If w=0.5, learning rate (η) =0.01, and gradient $\frac{\partial Error}{\partial w}$=0.1:

$$w=0.5-(0.01 \cdot 0.1) =0.5-0.001=0.499$$

- **Adam**:
  Adapts learning rates dynamically and uses momentum to accelerate convergence, making it highly effective for complex tasks. However, Adam requires more computational resources due to additional calculations for momentum and variance tracking.

- **Error Functions**

  The following are some common error functions.

  - **MSE (Mean Squared Error)**

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

    - **Example Calculation**:

      True value y=3.0, Predicted value $\hat{y}$=2.5

$$MSE = (3.0 - 2.5)^2 = 0.25$$

  - **MAE (Mean Absolute Error)**

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

    - **Example Calculation**:

      True value y=2.5, Predicted value $\hat{y}$=3.0

$$MAE = |2.5 - 3.0| = 0.5$$

- **Gradient and Training (Forward and Backpropagation)**

  - **Backpropagation**:

    - **Example Calculation (Single Neuron)**:

      Suppose the output $\hat{y}$ is predicted as 2.5, the true label y is 3.0, and the activation output is 0.4. The gradient of the weight w_1 is computed as:

$$\frac{\partial Error}{\partial w_1} = 2 \cdot (\hat{y}_i - y_i) \cdot \frac{\partial \hat{y}_i}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$

- Dataset information:
  1. cosine:
     - A simple cosine data, y = cos(x)
  2. iris(modified from https://www.kaggle.com/datasets/uciml/iris):
     - Predict which species of iris by given attributes: SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm
     - Species:
       - 0: setosa
       - 1: versicolor
       - 2: virginica
  3. housing(modified from https://www.kaggle.com/datasets/yasserh/housing-prices-dataset)
     - Predict the house price (in millions) by given attributes: bedrooms, bathrooms, sqft_living, sqft_lot, floors, waterfront, view, condition, grade, sqft_above, sqft_basement
  4. human_age(modified from https://www.kaggle.com/datasets/abdullah0a/human-age-prediction-synthetic-dataset)
     - Predict the human age (value is divided by 10) by given attributes: Gender, Height (cm) / 100, Weight (kg) / 100, Blood Pressure / 100, Diastolic / 100, Cholesterol Level (mg/dL) / 100, BMI / 100, Blood Glucose Level (mg/dL) / 100, Bone Density (g/cm2), Vision Sharpness, Hearing Ability (dB) / 100, Physical Activity Level, Smoking Status, Alcohol Consumption, Diet, Chronic Diseases, Medication Use, Family History, Cognitive Function / 100, Mental Health Status, Sleep Patterns, Stress Levels / 10, Pollution Exposure / 10, Sun Exposure / 10, Income Level
     - Gender:
       - 0: Male
       - 1: Female
     - Physical Activity Level:
       - 0: Low
       - 1: Moderate
       - 2: High
     - Smoking Status:
       - 0: Never
       - 1: Former
       - 2: Current
     - Alcohol Consumption:
       - 0: None
       - 1: Occasional
       - 2: Frequent
     - Diet:
       - 0: Vegetarian
       - 1: Low-carb

    - 2: Balanced
    - 3: High-fat
  - Chronic Diseases:
    - 0: None
    - 1: Diabetes
    - 2: Hypertension
    - 3: Heart Disease
  - Medication Use:
    - 0: None
    - 1: Occasional
    - 2: Regular
  - Family History:
    - 0: None
    - 1: Diabetes
    - 2: Hypertension
    - 3: Heart Disease
  - Mental Health Status:
    - 0: Poor
    - 1: Fair
    - 2: Good
    - 3: Excellent
  - Sleep Patterns:
    - 0: Insomnia
    - 1: Normal
    - 2: Excessive
  - Income Level:
    - 0: Low
    - 1: Medium
    - 2: High
5. cancer_type(modified from https://www.kaggle.com/datasets/erdemtaha/cancer-data)
  - Predict the cancer type by given attribute: radius_mean, texture_mean, perimeter_mean, area_mean, smoothness_mean, compactness_mean, concavity_mean, concave points_mean, symmetry_mean, fractal_dimension_mean, radius_se, texture_se, perimeter_se, area_se, smoothness_se, compactness_se, concavity_se, concave points_se, symmetry_se, fractal_dimension_se, radius_worst, texture_worst, perimeter_worst, area_worst, smoothness_worst, compactness_worst, concavity_worst, concave points_worst, symmetry_worst, fractal_dimension_worst
  - Diagnosis:
    - 0: M (***Malignant cancer***)

- 1: B (***Benign cancer)***